

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав(-ла):
Лук'янець В.В

Перевірив:
ас. Баранов О.М.

Конфігурація середовища

Варіант 1 (номер у журналі 11 → 1). ОС Windows 11 Pro; процесор Intel Core i3-12100F; ОЗП 15,8 ГБ; відеокарта AMD Radeon RX 6600 (чи використовувала її Ollama, окремо не перевірялося). Ollama 0.34.4, Python 3.14.3, бібліотека openai (клієнт OpenAI-сумісного API).

Локальні моделі: llama3.2:3b (2,0 ГБ, інструктивна) і qwen3:4b (2,5 ГБ, міркувальна). Хмарна модель: Groq, openai/gpt-oss-120b (~120 млрд параметрів, міркувальна). Модель llama-3.3-70b-versatile, запропонована в методичці, для використаного ключа недоступна (помилка 404 model_not_found), тому обрано найбільшу з доступних моделей провайдера. Ключ зберігається у файлі .env, який не потрапляє в репозиторій (.gitignore). Усі сирі результати: lab1.log, results.csv, quality.json.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпоінт /api/generate (скрипт task1_reasoning.py). Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів для кожної моделі зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ.
eval_count	200	5
Час, с	3,42	0,14
Токенів/с	62,9	104,6

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	3 / 5
eval_count	1385	183
Час, с	23,36	2,18

Типова помилка інструктивної моделі: у запусках 2 і 4 llama3.2:3b відповіла «50 грн» і «60 грн», неправильно склавши рівняння (замість різниці 100 грн підставила 10 грн). Крім того, у 4 з 5 запусків вона розв’язувала задачу російською,

хоча запит був українською. qwen3:4b у всіх п'яти запусках склала правильну систему рівнянь і зробила перевірку; в одному запуску (№4) також перейшла на російську.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь: У приховане поле thinking: з 200 згенерованих tokenів на питання про столицю лише 1–2 становлять саму відповідь «Київ», решта – «роздуми» (~600 символів самоперевірки). Вони генеруються й тарифікуються як звичайні вихідні токени, але користувач їх не бачить.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: На багатокрокових задачах: арифметика й текстові задачі, логіка, планування, код з перевіркою умов, де проміжні результати треба тримати «на папері». Невиправдані вони на простих фактах, переказі, класифікації й форматуванні, де відповідь очевидна, а важлива затримка.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивну навчають на парах «інструкція – відповідь» (SFT) і підлаштовують під уподобання людини (RLHF/DPO). Міркувальну додатково навчають з підкріпленням на задачах із перевірюваним результатом, винагороджуючи правильну кінцеву відповідь, тому вона вчиться будувати довгий ланцюжок міркувань із самоперевіркою.

4. Чи означає більша кількість tokenів, що модель «розумніша»?

Відповідь: Ні. На фактичному питанні 200 tokenів дали ту саму відповідь, що й 5. Довший вивід – це більший обсяг обчислень, який допомагає на певних задачах (у нас 5/5 проти 3/5), але не додає моделі знань.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: qwen3:4b (4 млрд параметрів, 2,5 ГБ) генерує ≈ 60 ток/с, Llama3.2:3b (3 млрд, 2,0 ГБ) – ≈ 86 –105 ток/с. Для кожного токена потрібно прочитати всі ваги, тож швидкість обмежена пропускнуою здатністю пам'яті й приблизно обернено пропорційна розміру моделі.

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, а витрати відрізняються у 40 разів за токенами (200 проти 5) і у ≈ 24 рази за часом (3,42 с проти 0,14 с). На задачі про ручку й зошит ситуація протилежна: qwen3:4b правильно розв’язала 5 із 5, а llama3.2:3b – лише 3 із 5, тобто розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 186,9 с проти 17,4 с ($8 \times 23,36$ с проти $8 \times 2,18$ с), що робить міркувальну модель некомфортною для інтерактивної роботи.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Питання варіанта 1: «Поясни різницю між списком і кортежем у Python.» Кожну з двох моделей запущено з двома system-повідомленнями (hello.py, temperature=0,7), по одній відповіді.

Таблиця 3 – Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	1912	українська з російськими й англійськими вкрапленнями	≈ 15 (розділи, списки, код)	Ні – відповідь розлога
llama3.2:3b	українською, 2 речення	193	українська зі спотвореними словами («accessивувати»)	2	Частково – довжину дотримано, мову зіпсовано
qwen3:4b	лаконічний асистент	978	українська	≈ 10 (нумерований список + приклад)	Частково – стисліше, але не «лаконічно»
qwen3:4b	українською, 2 речення	280	українська	2	Так

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	54	665	719
llama3.2:3b	українською, 2 речення	58	62	120
qwen3:4b	лаконічний асистент	55	1438	1493

qwen3:4b	українською, 2 речення	64	967	1031
----------	------------------------	----	-----	------

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: Обидві вклались у два речення, але інструкцію в цілому точніше виконала qwen3:4b: llama3.2:3b при цьому зіпсувала мову відповіді.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: Повністю іншою мовою – ні, але llama3.2:3b вставила в українську відповідь англійсько-українські «гібриди» (accessикувати), а у відповіді на «лаконічний» system – російське «Однако» й англійське ONLY.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Відповідь: Зріс: 54 → 58 у llama3.2:3b і 55 → 64 у qwen3:4b, бо друге system-повідомлення довше, а українські слова розбиваються токенизатором на більше tokenів (різниця між моделями – через різні токенизатори).

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: До completion_tokens входять і приховані токени міркування: qwen3:4b видала 967 tokenів заради відповіді у 280 символів, а llama3.2:3b – 62 токени за 193 символи.

Відповіді чат-бота на питання кроку 7

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови?

Відповідь: Вони відрізняються обсягом і якістю донавчання на інструкціях, розміром та часткою української в даних. Міркувальна модель може спершу «спланувати» відповідь і звірити її з обмеженнями, а мала інструктивна генерує за один прохід і легше губить одне з обмежень.

2. Чим system відрізняється від user, чи справді він має більший пріоритет?

Відповідь: Для моделі обидва – текст у контексті, але system позначено окремою роллю, і модель навчена надавати їй більшу вагу («ієрархія інструкцій»). Це навчена схильність, а не жорстке правило, тому пріоритет не гарантований.

3. Чому міркувальна модель витрачає більше вихідних токенів навіть при однаковій видимій відповіді?

Відповідь: Тому що вона спочатку генерує прихований хід думок, який рахується у completion_tokens і оплачується, хоча в content не потрапляє.

4. Що станеться при суперечливих system і user?

Відповідь: Зазвичай модель віддасть перевагу system, але це залежить від моделі й формулювань: вона може виконати user-інструкцію або видати компромісну відповідь. Результат треба перевіряти, а не покладатися на правило.

5. Чому малі моделі гірше дотримуються інструкцій щодо мови?

Відповідь: В їхньому навчанні українських текстів мало порівняно з англійськими, а малий розмір не дозволяє одночасно втримувати кілька обмежень (мова, довжина, стиль), тому вони зісковзують до домінантної мови або змішують їх.

Висновок за завданням 2

System-повідомлення – найдешевший спосіб керувати поведінкою моделі: воно помітно змінило довжину (1912 → 193 символи у llama3.2:3b, 978 → 280 у qwen3:4b), але надійним його назвати не можна. Мала інструктивна модель виконала обмеження довжини, проте зіпсувала мову, а інструкція «лаконічний» взагалі не стримала обидві моделі. Одного тексту інструкції недостатньо для гарантованого формату; якщо модель її ігнорує, варто конкретизувати вимогу (числа, приклад формату), перенести її ближче до кінця запиту, знизити temperature, взяти більшу модель і, головне, перевіряти результат програмно (валідація й повторний запит).

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Питання варіанта 1: «Поясни різницю між списком і кортежем у Python. Коротко.» (compare.py, temperature=0,7). Для переходу між джерелами змінюється лише запис у словнику PROVIDERS.

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	openai/gpt-oss-120b (Groq)
Час відповіді, с	7,61	1,49
completion_tokens	438	517 (з них 39 – міркування)
Довжина відповіді, символів	1198	1302
Мова відповіді	українська зі спотвореними словами та іншомовними вкрапленнями (réfécнції, перushhata, operacje)	українська, природна
Фактичних помилок у відповіді	щонайменше 4	1

Помилки локальної моделі: стверджує, що елементи кортежа «можна додати та видалити за допомогою методів»; приписує спискам операції «-» та «/»; у прикладі виконує `my_list[:2] + (4,)`, що дає `TypeError` (список + кортеж); вигадує «зміну кортежа за допомогою сценаріїв». Помилка хмарної: у таблиці про можливість бути ключем словника для списку стоїть «Не може, бо незмінний» (має бути «змінний»).

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: У `compare.py` – жодного: цикл однаковий для обох. Відмінності зібрано в словнику `PROVIDERS`: три значення (`base_url`, `api_key` з `.env`, `model`) в записі «cloud».

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна: вона подала таблицю з мутованістю, синтаксисом, пам'яттю, швидкістю й тим, що кортеж можна використати як ключ словника. Локальна натомість стверджує, що до кортежа можна додавати елементи «за допомогою методів», і наводить код, що падає з `TypeError`.

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, і те й інше: іншомовні вкраплення в слова (réfécнції, перushhata, operacje), кальки («кортежи») і щонайменше чотири фактичні помилки, перелічені вище.

4. Хмарна модель відповіла швидше за локальну – за рахунок чого, якщо запит летить через інтернет?

Відповідь: Мережева затримка становить десятки–сотні мілісекунд, а генерація – секунди. Хмара виграє за рахунок спеціалізованого серверного обладнання (у Groq – власні прискорювачі) і того, що gpt-oss-120b є архітектурою Mixture-of-Experts, де на кожен токен активна лише частина параметрів; локальна модель рахується на побутовому залізі.

Відповіді чат-бота на питання кроку 6

1. Чому формат запитів OpenAI став стандартом?

Відповідь: OpenAI першою масово запустила API чат-моделей, навколо нього вирости SDK та інструменти. Сумісність знижує вартість переходу для розробників, тому інші провайдери (Ollama, Groq, Google) додали сумісні ендпойнти.

2. Наскільки розмір моделі впливає на якість порівняно з іншими факторами?

Відповідь: Розмір сильно впливає на фактичні знання й багатомовність, але якість визначають також дані й донавчання (instruct/RLHF), архітектура, квантизація та промпт. Наша пара відрізняється не лише розміром (3 млрд проти ~120 млрд), а й способом навчання, тому вплив розміру в чистому вигляді виміряти не можна.

3. Чим пояснюється, що хмарна відповіді швидше попри затримку мережі?

Відповідь: Серверні GPU/спеціалізовані чипи мають на порядок вищу пропускну здатність пам'яті, ніж локальний комп'ютер, а затримка мережі мала порівняно з часом генерації.

4. Які ризики для приватності і що з цим роблять у комерційних проєктах?

Відповідь: Текст запиту потрапляє до третьої сторони: його можуть логувати, зберігати чи використовувати. У комерційних проєктах укладають угоди про

обробку даних, обирають тарифи без навчання на даних і регіон обробки, маскують персональні дані або тримають чутливі задачі на локальних чи власних моделях.

5. Що таке rate limit у безкоштовних тарифах і як його обходять?

Відповідь: Це обмеження кількості запитів і токенів за хвилину/добу; при перевищенні сервер повертає 429. Розробники застосовують черги та повтори з експоненційною затримкою, кешування, пакетну обробку, резервних провайдерів або переходять на платний тариф.

Висновок за завданням 3

Локальну модель я обрав би, коли дані не можна відправляти назовні (персональні, комерційно чутливі), потрібна робота без інтернету або великий обсяг простих запитів, де важлива нульова вартість; хмарну – коли потрібні якість, точність фактів і швидкість, а тексти не конфіденційні. На нашому прикладі різниця в якості очевидна: 3B-модель вигадувала API-поведінку й псувала слова, а 120B-модель дала майже безпомилкову відповідь у 5 разів швидше.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Промпти варіанта 1: короткий «Столиця Франції?», середній «Поясни в одному абзаці, що таке рекурсія.», довгий «Напиши інструкцію з 10 пунктів, як почати вивчати Python.» Застосовано три правила: прогрів, три повтори з медіаною, без паралельних запитів (benchmark.py, стрімінг). TTFT вимірювано до першого фрагмента видимого тексту (delta.content), тобто для міркувальної хмарної моделі він включає короткий прихований етап міркування.

Таблиця 7 – Результати вимірювання швидкості (медіана з трьох повторів)

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2,066	2,224	11,5
локальна	llama3.2:3b	середній	2,084	2,976	80,7
локальна	llama3.2:3b	довгий	2,118	11,199	207,9
хмарна	gpt-oss-120b	короткий	0,333	0,470	113,1
хмарна	gpt-oss-120b	середній	0,795	1,056	381,5
хмарна	gpt-oss-120b	довгий	0,361	2,553	1168,8

Примітка: для короткого промпту показник «симв./с» малоінформативний, бо відповідь складається з кількох слів і час майже цілком становить TTFT.

Таблиця 8 – Холодний і теплий старт локальної моделі (llama3.2:3b, короткий промпт)

Показник	Холодний старт	Теплий старт
TTFT, с	7,292	2,113
Всього, с	7,405	2,202

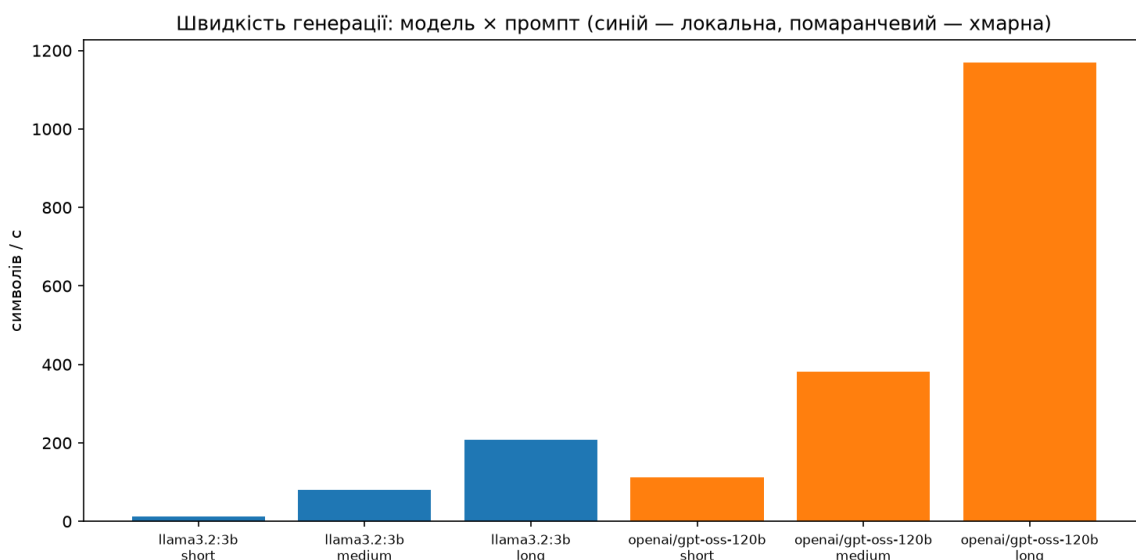


Рисунок 1 – Швидкість генерації в символах за секунду для шести комбінацій «модель × промпт»

Відповіді на питання (крок 6)

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: У хмарної gpt-oss-120b: 0,33–0,80 с проти 2,07–2,12 с у локальній. Серверне обладнання швидше обробляє запит, а мережева затримка мала порівняно з різницею.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»?

Відповідь: Так: хмарна модель виграла за обома показниками. Припущення з методички про гірший TTFT хмари в цих вимірах не підтвердилося.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Відповідь: Хмара швидша в усіх випадках. За загальним часом розрив $4,7\times$ (короткий), $2,8\times$ (середній) і $4,4\times$ (довгий); за швидкістю генерації – $9,8\times$, $4,7\times$ і

5,6×, тож на довгій відповіді вирішальною стає саме швидкість генерації, а не TTFT.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Локальна модель стабільна за TTFT (2,06–2,13 с), але швидкість генерації коливається до 45% (207,9 проти 179,5–260,0 симв./с на довгому промпті, бо довжина відповіді щоразу різна). У хмарної TTFT коливається до 3,4 разів (0,36–1,22 с на середньому промпті). Одного запуску недостатньо.

5. Чому холодний старт повільніший за теплий?

Відповідь: При холодному старті Ollama спершу завантажує ваги моделі з диска в пам'ять (TTFT 7,29 с проти 2,11 с), а в теплому вони вже там.

Відповіді чат-бота на питання кроку 7

1. Що відбувається під час TTFT і чому його вимірюють окремо?

Відповідь: Запит токенізується, вся вхідна послідовність обробляється за один прохід (prefill), будується KV-кеш і генерується перший токен. Це час, коли користувач бачить порожній екран, тоді як подальша генерація – окрема фаза, що масштабується з довжиною відповіді.

2. Чому хмара може мати гірший TTFT, але кращу швидкість генерації?

Відповідь: До TTFT додаються мережа, маршрутизація й черга запитів, а швидкість генерації визначається потужним обладнанням із великою пропускнуою здатністю пам'яті.

3. Чому перший запуск локальної моделі повільніший?

Відповідь: Це холодний старт: ваги завантажуються з диска в оперативну (або відео-) пам'ять і ініціалізується кеш; наступні запити використовують уже завантажену модель.

4. Чому беруть медіану кількох запусків?

Відповідь: Одиначні виміри спотворюють випадкові чинники (мережа, фонові процеси, кеші), а медіана стійка до викидів.

5. Чому обробка вхідного запиту йде швидше, ніж генерація такої ж довжини?

Відповідь: Prefill обробляє всі токени пром프트 паралельно за один прохід, а генерація послідовна – один токен за прохід із повторним читанням усіх ваг.

Висновок за завданням 4

Для інтерактивного чат-бота важливіший TTFT, бо саме він визначає, скільки користувач дивиться на порожній екран; потім текст може «друкуватися» в темпі читання. Для фонові обробки великих обсягів тексту TTFT не має значення, а вирішують пропускну здатність (символів або токенів за секунду) і вартість. Окрема практична порада: холодний старт локальної моделі (7,3 с проти 2,1 с) варто приховувати попереднім прогрівом.

Завдання 5. Оцінювання якості моделей за рубрикою

Набір завдань варіанта 1 (quality.py) складено до перегляду відповідей: рік заснування Одеського національного університету імені І. І. Мечникова (правильна відповідь – 1865 рік, за uk.wikipedia.org та офіційними даними університету), JSON name/year/city, стиснення тексту з шістьма числами (1865, 12, 14500, 3500, 179, 105 – текст із Вікіпедії), is_palindrome(s), задача про 25 студентів (правильна відповідь – 8), пояснення різниці між стеком і чергою. Кожне завдання прогнано двічі. Правило підсумкового бала: для завдання 1 – за таблицею критеріїв (якщо прогони розійшлися, максимум 1 бал), для решти – нижчий із двох прогонів. Бали виставлено мною за прочитаними відповідями, завдання 2 перевірено через json.loads() (check_json.py).

Таблиця 13 – Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна (gpt-oss-120b)
1. Факт	0	0	1
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	2	2	2
5. Логіка	0	2	2
6. Українська	0	1	2
Разом /12	4	9	11

Таблиця 14 – Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна (gpt-oss-120b)
1. Факт	так (1908 / 1921)	так (1804 / 1900)	так (1865 / 1902)
2. Формат	ні	ні	ні
3. Стиснення	ні	ні	ні
4. Код	ні	ні	ні

5. Логіка	так (0 / 2)	ні	ні
6. Українська	ні	так (1 / 2)	ні

Пояснення до оцінок. Завдання 1: усі три моделі у двох прогонах дали різні роки, тому це вигадка (правильно 1865 лише в одному прогоні хмарної, що за рубрикою дає 1 бал). Завдання 2: усі 6 відповідей парсяться `json.loads()` без правок (2 бали), проте значення `year` у 5 з 6 відповідей неправильні (1940, 1919, 1917, 1991, 1900; правильно 1865 лише в одному прогоні хмарної). Завдання 3: `llama3.2:3b` в обох прогонах втратила числа (прогін 1: 3500, 179, 105; прогін 2: 14500, 179, 105), тому 0; `qwen3:4b` і хмарна зберегли всі шість чисел у двох реченнях. Завдання 4: усі функції коректні; хмарна обгорнула код у `markdown`-блок, але без пояснень, тому 2 бали. Завдання 5: у прогоні 1 `llama3.2:3b` видала безглузду «комплексну число» без відповіді (0), у прогоні 2 правильно отримала 8 (2). Завдання 6: `llama3.2:3b` в обох прогонах вигала про «хворобу» й «скло» (0); у `qwen3:4b` зустрічаються огріхи («Основне відмінність», «зворотньому», «лінію чеку»), тому 1; хмарна – природна українська (2). Оцінка мови суб’єктивна.

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і на яких завданнях виникла основна різниця?

Відповідь: Хмарна `gpt-oss-120b` – 11 з 12, далі `qwen3:4b` – 9, `llama3.2:3b` – 4. Основна різниця виникла в завданнях 3 (збереження чисел), 5 (логіка) та 6 (українська), де `llama3.2:3b` отримала 0.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: У завданні 1 – в усіх трьох моделях (вигадують роки), у завданні 5 – у `llama3.2:3b` (0 і 2), у завданні 6 – у `qwen3:4b` (1 і 2). Розбіжність між прогонами означає, що результат залежить від випадкового вибору, тож довіряти одному запуску не можна.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, усі три в обох прогонах віддали чистий JSON, який одразу парситься. Проблеми з обгорткою я не спостерігав (на вибірці з двох прогонів), проблема була у вигаданих значеннях, а не у форматі.

4. Чи були проблеми з українською мовою?

Відповідь: Так. Llama3.2:3b змішувала мови й вставляла іншомовні слова, з російською розв'язувала задачу про ручку. qwen3:4b мала окремі огріхи («лінія чеку»), а в одному прогоні задачі про ручку перейшла на російську. Хмарна була найприроднішою (лише «лінія в магазині» як калька).

Крок 7. Приклад провалу (Llama3.2:3b, завдання 6, прогін 1)

Повний текст запиту:

Чим стек відрізняється від черги. Поясни українською, два абзаци.

Повний текст відповіді, без скорочень:

Stidak Glass та Choroba Glass (також відомий як Choroba Stakhanovita) – дві різні хвороби, які призводяться для відрізу від черги. Один з них – Choroba Stakhanovita, яка характеризується підвищеним рентгенопровідністю стовпців.

Ця хвороба є рідкістю і характерна для людей з підвищеним генетическим ризиком.

Правильна відповідь: стек – структура даних, що працює за принципом LIFO (останнім прийшов – першим вийшов), елементи додаються й вилучаються з одного кінця; черга – за принципом FIFO (першим прийшов – першим вийшов), додавання в кінець, видалення з початку.

Пояснення. Модель не впізнала слово «стек» як термін інформатики: в українських текстах воно рідше зустрічається саме в цьому значенні, а схожість із російським «стекло» штовхнула генерацію в бік «скла» та «хвороби Glass». Далі спрацював механізм породження найправдоподібнішого продовження: кожен вигаданий токен ставав контекстом для наступних, і відповідь дедалі більше віддалялася від теми, прикрашаючись вигаданими «фактами» («Choroba Stakhanovita»). Дрібна модель із малою часткою українських даних не мала запасу знань, щоб виправити початковий хибний вибір, а впевнений тон відповіді не дозволяє відрізнити вигадку від знання.

Відповіді чат-бота на питання кроку 8

1. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: Стандартизовані бенчмарки (MMLU, GSM8K, HumanEval), оцінювання іншою моделлю (LLM-as-a-judge), парні порівняння людьми з рейтингом Ело (Chatbot Arena), автоматичні метрики (BLEU, ROUGE, BERTScore), юніт-тести для коду й спеціальні набори на галюцинації та безпеку.

2. Чому моделі часто ігнорують інструкцію не використовувати markdown при генерації JSON?

Відповідь: У навчальних даних код і JSON майже завжди оформлені markdown-блоками, а чат-донавчання закріплює цю звичку. Інструкція конкурує з сильним статистичним шаблоном. (У моєму прогоні всі моделі формат витримали, але на двох спробах це нічого не гарантує.)

3. Чому малі моделі гірше пишуть українською?

Відповідь: Українських текстів у навчанні значно менше, ніж англійських, токенизація українських слів менш ефективна, а малий обсяг параметрів залишає менше ємності для «слабких» мов.

4. Чому та сама модель на однаковий запит дає різні відповіді?

Відповідь: Токени вибираються випадково відповідно до ймовірностей ($\text{temperature} > 0$), а навіть при $\text{temperature}=0$ дрібні відмінності обчислень із плаваючою комою й паралелізації можуть змінити вибір.

5. Що таке галюцинація і чому модель радше вигадас факт, ніж визнає незнання?

Відповідь: Це правдоподібна, але хибна відповідь. Модель оптимізована генерувати ймовірне продовження, а не перевіряти знання, а навчальні дані й оцінювання рідко заохочують відповідь «не знаю».

Висновок за завданням 5

Оцінювання за рубрикою дало відтворюваний профіль моделей (4, 9 та 11 балів із 12), але вибірка мала: два прогони на завдання, шість завдань, а оцінку завдань 5 і 6 частково визначає суб'єктивне судження. Найпоказовішими виявились завдання 1 (усі моделі вигадують роки), 3 (перевіряє збереження чисел) і 6 (мова); завдання 2 у цьому прогоні не розрізнило моделі, бо всі витримали формат. Для роботи з українськомовним текстом я обрав би хмарну gpt-oss-120b,

якщо дані не конфіденційні, а серед локальних – qwen3:4b (9/12), яка помітно надійніша за llama3.2:3b.

Завдання 6. Керування випадковістю: temperature і top_p

Варіант 1: творче завдання «Назва стартапу в галузі логістики», фактичне питання «Рік заснування Національного університету «Одеська Політехніка»». Правильна відповідь – 1918 рік (університет відкрито 18 вересня 1918 року; статус національного отримав у 2021 році). Джерело: https://uk.wikipedia.org/wiki/Одеська_політехніка. Експерименти виконано на llama3.2:3b (sampling.py). «Унікальних» означає кількість різних текстів відповідей із п'яти.

Таблиця 16 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	2	0 (усі 1992)
temperature	0.7	5	—
temperature	1.5	5	0 (2005, 1980, 1997, дві безглузді)
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 17 – Перевірка відтворюваності (фактичне питання)

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Примітка: на творчому завданні при temperature=0,0 виявилось 2 унікальні відповіді з 5: чотири тексти збігаються, а один відрізняється кінцевим реченням, тобто повної детермінованості temperature=0 не гарантує. Також модель у творчому завданні замість однієї назви частіше видавала вступну фразу й список, тому метрика «унікальних» тут рахує розбіжність текстів, а не назв.

Відповіді на питання (крок 7)

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: 0 (або $\leq 0,2$): потрібні передбачуваність і стабільний формат, а випадковість лише збільшує ризик зіпсованого JSON.

2. Яке – генератору варіантів заголовків?

Відповідь: Близько 0,8–1,0: різноманітність потрібна, але без розпаду мови. Мій експеримент це підтверджує: при 0,7 – 5 різних відповідей без сміття, при 1,5 з'являється змішування мов.

3. Чим вплив `top_p` відрізняється від впливу `temperature` за характером, а не за силою?

Відповідь: Temperature змінює форму всього розподілу (при високому значенні підвищує шанс рідкісних токенів), а `top_p` лише відсікає «хвіст» і не змінює співвідношення ймовірностей решти. У мене `top_p=0,1` дав одну й ту саму відповідь п'ять разів, а `top_p=1,0` – п'ять різних, тоді як `temperature=1,5` додатково породив «сміття» з інших мов.

4. Чи означає `temperature=0` повну відтворюваність результату?

Відповідь: Ні. На фактичному питанні 10 запусків дали одну відповідь, проте на творчому завданні при `temperature=0` з'явилося 2 різні тексти з 5. Відмінності обчислень із плаваючою комою та паралелізація можуть давати розбіжності.

5. Наведіть найбезглуздішу відповідь при `temperature=1,5` і поясніть, чому вона могла з'явитися.

Відповідь: «Національний університет по імені Івана Шеноні та імен Миколади Політехніку було засновано у 1997 році.» При високій температурі розподіл ймовірностей сплющується, рідкісні токени (вигадані імена, слова з інших мов) отримують помітний шанс, а кожен вибраний токен стає контекстом, що закріплює помилку.

Відповіді чат-бота на питання кроку 8

1. Що математично робить `temperature` із розподілом ймовірностей?

Відповідь: Логіти ділять на T перед softmax: $p_i \propto \exp(z_i / T)$. При $T \rightarrow 0$ розподіл вироджується в $\arg\max$, при $T > 1$ стає плоскішим.

2. Чим `top_p` відрізняється від `temperature` за механізмом?

Відповідь: Top_p (nucleus sampling) сортує токени за ймовірністю, бере мінімальну множину із сумарною ймовірністю $\geq p$, перенормовує її й вибирає лише з неї. Temperature змінює сам розподіл, top_p обрізає його.

3. Чи є сенс змінювати обидва параметри одночасно?

Відповідь: Зазвичай радять налаштовувати один із них: їхні ефекти накладаються, і результат важко інтерпретувати.

4. Чому навіть при temperature=0 відповідь не гарантовано однакова?

Відповідь: Обчислення з плаваючою комою не асоціативні, а паралельні редукції на GPU/CPU, пакетування запитів і оновлення бібліотек можуть змінити останні розряди й відповідно вибір токена.

5. Які значення temperature використовують на практиці?

Відповідь: Для структурованих даних та фактів – 0–0,2, для діалогу – близько 0,7, для творчих завдань – 0,8–1,2.

Висновок за завданням 6

Для наступних лабораторних, де модель має повертати структуровані дані, я поставлю temperature=0 (або $\leq 0,2$) і залишу top_p за замовчуванням, а формат гарантуватиму валідацією й повторним запитом. Значення «за замовчуванням» (у методичці використано 0,7) може виявитись непридатним, бо додає випадковості там, де потрібна відтворюваність, а експеримент показав і інше: температура не робить модель розумнішою – при temperature=0 llama3.2:3b стабільно, 10 разів із 10, повторювала неправильну відповідь 1992, тобто послідовність не дорівнює правильності.

Підсумкова таблиця порівняння моделей

Таблиця 18 – Підсумкове порівняння моделей

Критерій	Локальна мала (llama3.2:3b)	Локальна більша (qwen3:4b)	Хмарна (gpt-oss-120b, Groq)
Швидкість генерації	≈86–105 ток/с; 208–240 симв./с на довгій відповіді	≈58–63 ток/с; довгі роздуми (у середньому 1385 токенів і 23 с на просту задачу)	≈1100–1200 симв./с на довгій відповіді
TTFT	2,1 с (теплий), 7,3 с (холодний)	окремо не вимірювався; мінімальний запит займає 3,4 с через роздуми	0,33–0,80 с
Якість, балів /12	4	9	11
Стабільність формату (завдання 2)	2 з 2 парсяться, але значення вигадані	2 з 2 парсяться, значення вигадані	2 з 2 парсяться, в одному прогоні значення вірне
Потрібні ресурси	2,0 ГБ на диску, ОЗП/відеопам'ять, встановлена Ollama	2,5 ГБ на диску, більше часу й енергії на відповідь	інтернет і API-ключ; локальних ресурсів майже не потрібно

Вартість	0 (лише електроенергія)	0 (лише електроенергія)	безкоштовний тариф з лімітами, далі оплата за токени
Куди йдуть ваші дані	залишаються на вашому комп'ютері	залишаються на вашому комп'ютері	на сервери провайдера (Groq)
Працює без інтернету	так	так	ні
Коли обирати саме її	Прості, швидкі й недорогі задачі з конфіденційними даними чи без мережі: класифікація, витяг за шаблоном, переказ наданого тексту. Не для фактів і української.	Локальні задачі, де потрібні міркування й краща мова, а затримка в десятки секунд припустима: аналіз, розрахунки, фонові обробка без передачі даних назовні.	Коли потрібні якість, точність, українська мова й швидкість, а дані не конфіденційні та інтернет стабільний; як резерв – перемикання на локальну модель.

Правило вибору: спочатку відповідаю на питання про дані. Якщо їх не можна передавати третій стороні або немає інтернету – локальна модель, причому для простих задач llama3.2:3b, для міркувань і української – qwen3:4b, з обов'язковою програмною перевіркою результату. Якщо дані відкриті, а потрібні факти, українська мова, швидка відповідь чи складне міркування – хмарна модель, з урахуванням rate limit і резервного провайдера.

Загальні висновки

1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребував жодної зміни в compare.py – лише запису в словнику PROVIDERS; це основа стратегії резервування провайдерів у реальних проєктах;

2) Затримка складається з двох незалежних показників – TTFT та швидкості генерації; у наших вимірах хмара виграла за обома (0,33–0,80 с проти $\approx 2,1$ с і $\approx 5\times$ за швидкістю), але одиничний вимір ненадійний: TTFT хмарної моделі коливався до 3,4 разів між повторами;

3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні (5/5 проти 3/5) і є чистою перевитратою на фактичних питаннях (40× токенів для тієї самої відповіді);

4) Мала локальна модель ненадійна для фактів, чисел і української мови (4 бали з 12), а всі моделі вигадували роки; для фактів потрібні джерела (RAG), а для будь-якого результату – програмна перевірка. У цьому прогоні всі моделі витримали формат JSON, але це два прогони, і надійність формату промптом не гарантується;

5) Мовні моделі недетерміновані: навіть при $temperature=0$ траплялися різні відповіді, а $temperature=0$ не гарантує правильності (10 разів з 10 однакова хибна відповідь 1992). Тому будь-яке вимірювання потребує повторів, а порівняння двох версій системи – врахування шуму.